

Week 4

Team project update/progress

Table of Contents

- 01 Weekly Progress Summary
- 02 Implementation Task Division
- 03 Implementation Updates (from each member)
- 04 Next Steps

Weekly Progress Summary

Area	Progress Made
Tool Running	All team members were able to run the tool; the main issue left was API tokens.
UI Design	Finished the UI design using the IntelliJ Figma Kit.
UI Panel Review	Planned to use runIDE and Dr. AlOmar's demo video to compare the current UI Panel with the Figma design.
Implementation	Divided UI implementation tasks and agreed on the GitHub branch / pull request workflow.
Execution	Worked individually throughout the week and on separate branches that we will review and merge to the main branch when ready.

Implementation Task Division

Member	UI Section	Main Responsibility
Jake	Multiple clone UI	Implement separate clone cards and allow the user to decide on each clone individually.
Amaro	Explanation section	Show clone type, file location, pasted code location, line numbers, and suggested method.
Juliana	Diff preview	Show original duplicate code and extracted replacement.
Dhir	AI Trust / validation	Show Clone Detection, Refactoring Agent, Usefulness, Compilation, and Test/Behavior status.
Fin	Buttons / actions	Apply, Reject/Cancel, Open in editor, View details, and disabled Apply state when validation fails.

Goal: everyone owns one clear section so implementation work can happen in parallel.

Multiple Clone UI Decision

Separate Cards

When more than one clone is found, each clone should appear in its own card.

Individual Choice

The UI asks the user if they want to refactor each specific clone.

Clear Actions

Each card has Exclude and Include actions to decide if it should be refactored or not.



Select Clone Occurrence

Clone occurrence: 1/2

Occurrence 1: MethodName [Lines: #-#]

This clone occurrence has been highlighted.

Include this occurrence in the refactoring?

Cancel

Exclude

Include

Explanation Section

01

Make the explanation section easier for the developer to understand.

02

Clearly show the clone type, file locations, method names, and line numbers.

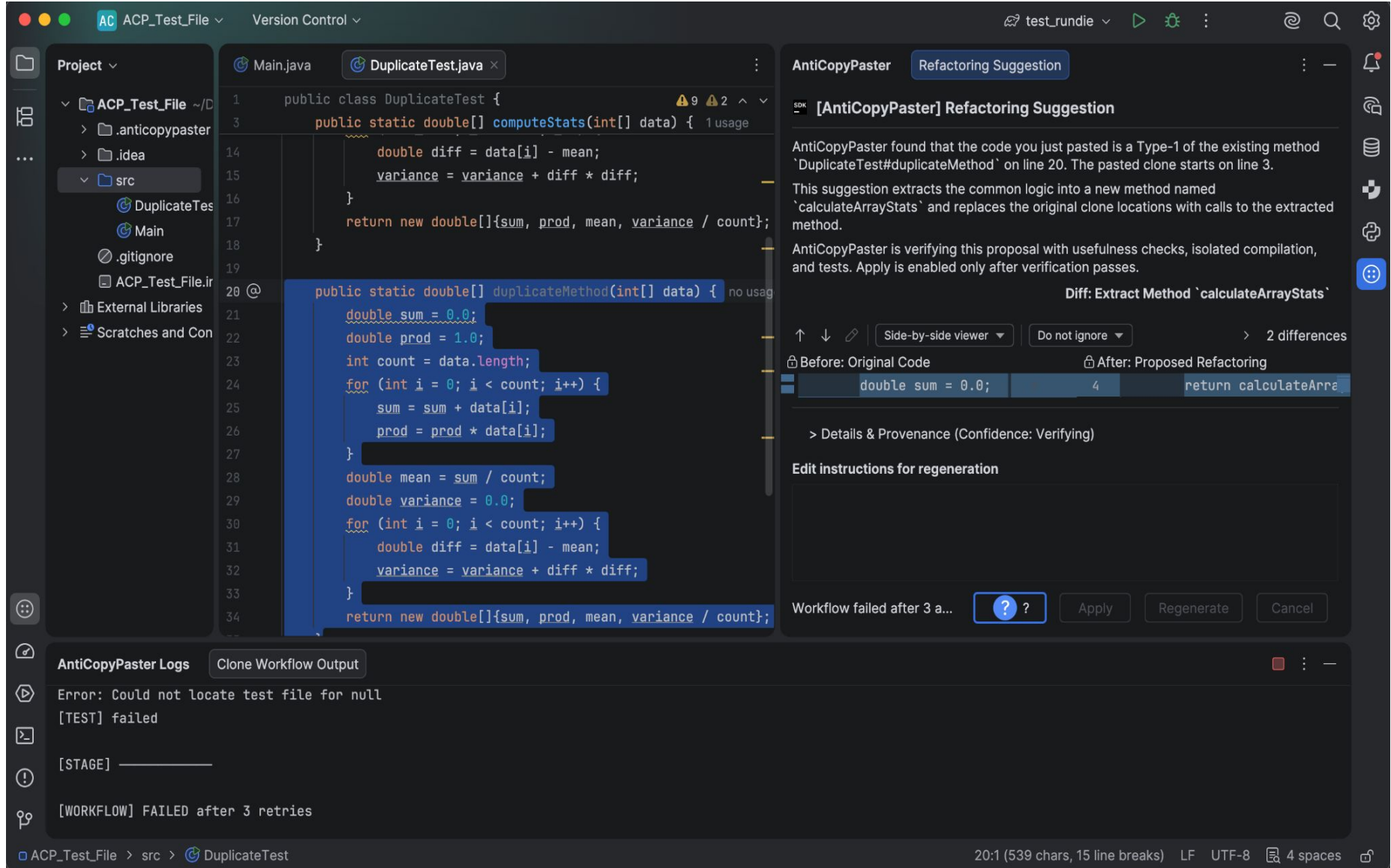
03

Explain the suggested Extract Method refactoring in simple terms.

04

Help the user quickly decide whether to inspect, apply, or reject the refactoring.

Original



Updated

Project ▾
ACP_Test_File ~/
 .anticopypaster
 .idea
 src
 DuplicateTes
 Main
 .gitignore
 ACP_Test_File.ir
External Libraries
Scratches and Con

ACP_Test_File ▾
src ▾
DuplicateTest

Main.java DuplicateTest.java ×

```
1 public class DuplicateTest {  
3     public static double[] computeStats(int[] data) { 1 usage  
14         double diff = data[i] - mean;  
15         variance = variance + diff * diff;  
16     }  
17     return new double[]{sum, prod, mean, variance / count};  
18 }  
19  
20 @ public static double[] duplicateMethod(int[] data) { no usag  
21     double sum = 0.0;  
22     double prod = 1.0;  
23     int count = data.length;  
24     for (int i = 0; i < count; i++) {  
25         sum = sum + data[i];  
26         prod = prod * data[i];  
27     }  
28     double mean = sum / count;  
29     double variance = 0.0;  
30     for (int i = 0; i < count; i++) {  
31         double diff = data[i] - mean;  
32         variance = variance + diff * diff;  
33     }  
34     return new double[]{sum, prod, mean, variance / count};  
35 }
```

AntiCopyPaster Refactoring Suggestion

[AntiCopyPaster] Refactoring Suggestion

AntiCopyPaster found a Type-1 clone in this file.
The pasted code in `DuplicateTest#computeStats` starts on line 3 and matches existing code in `DuplicateTest#duplicateMethod` starting on line 20.

Suggested refactoring:
Extract the shared calculation logic into a new helper method named `calculateArrayStats`.
This would replace both duplicate code blocks with calls to the new method, reducing repeated code while keeping the same behavior.

Verification status:
AntiCopyPaster is checking usefulness, isolated compilation, and tests.
The Apply button will stay disabled until verification passes.

Diff: Extract Method `calculateArrayStats`

↑ ↓ ↻ Side-by-side viewer ▾ Do not ignore ▾ > 2 differences

Before: Original Code	After: Proposed Refactoring
double sum = 0.0;	4 return calculateArra

> Details & Provenance (Confidence: Verifying)

Edit instructions for regeneration

Workflow failed after 3 a... ? ? Apply Regenerate Cancel

AntiCopyPaster Logs Clone Workflow Output

Error: Could not locate test file for null
[TEST] failed
[STAGE] _____
[WORKFLOW] FAILED after 3 retries

ACP_Test_File > src > DuplicateTest 20:1 (539 chars, 15 line breaks) LF UTF-8 4 spaces

v Details & Provenance (Confidence: Verifying)

Clone Type:

Type-1 (same code except for whitespace or comments)

Source Location:

DuplicateTest.java:20 (DuplicateTest#duplicateMethod)

Pasted Location:

DuplicateTest.java:3 (DuplicateTest#computeStats)

Clone ID:

c1_selected

Target Methods:

DuplicateTest#computeStats <-> DuplicateTest#duplicateMethod

Suggested Refactor:

extracted_method

Selected Ranges:

3-18, 20-35

Detection Reason:

The code block inside computeStats is identical to the code block inside duplicateMethod, performing the same statistical calculations on the input array.

Verification:

Usefulness check, isolated compilation, and tests are running.

Edit instructions for regeneration

Workflow failed after 3 attempts. See the logs below for details.

?

?

Apply

Regenerate

Cancel

v Details & Provenance (Confidence: Verifying)

Clone Type:

Type-1 clone — same code except for whitespace or comments

Existing Clone Location:

DuplicateTest.java:20–35 (DuplicateTest#duplicateMethod)

Pasted Code Location:

DuplicateTest.java:3–18 (DuplicateTest#computeStats)

Clone ID:

c1_selected

Target Methods:

DuplicateTest#computeStats <-> DuplicateTest#duplicateMethod

Suggested Refactoring:

Extract Method

Suggested Method Name:

calculateArrayStats

Selected Ranges:

Lines 3–18 and 20–35

Detection Reason:

Both methods contain the same array-statistics logic for sum, product, mean, and variance.

Verification:

Usefulness check, isolated compilation, and tests are running.

Edit instructions for regeneration

Workflow failed after 3 attempts. See the logs below for details.

?

?

Apply

Regenerate

Cancel

Original

AntiCopyPaster

Refactoring Suggestion

SDK

[AntiCopyPaster] Refactoring Suggestion

AntiCopyPaster found that the code you just pasted is a Type-1 of the existing method `DuplicateTest#duplicateMethod` on line 20. The pasted clone starts on line 3. This suggestion extracts the common logic into a new method named `calculateArrayStats` and replaces the original clone locations with calls to the extracted method. AntiCopyPaster is verifying this proposal with usefulness checks, isolated compilation, and tests. Apply is enabled only after verification passes.

Diff: Extract Method `calculateArrayStats`

↑

↓

✂

Side-by-side viewer

Do not ignore

Highlight words

✕

↕

⚙

?

2 differences

⚙

Before: Original Code

// Target clone methods
// DuplicateTest#computeStats
public static double[] computeStats(int[] data) {
 double sum = 0.0;
 double prod = 1.0;
 int count = data.length;
 for (int i = 0; i < count; i++) {
 sum = sum + data[i];
 prod = prod * data[i];
 }
 double mean = sum / count;
 double variance = 0.0;
}

After: Proposed Refactoring

1 // Target clone methods
2 // DuplicateTest#computeStats
3 public static double[] computeStats(int[] data) {
4 return calculateArrayStats(data);
5 }
6
7 // DuplicateTest#duplicateMethod
8 public static double[] duplicateMethod(int[] data) {
9 return calculateArrayStats(data);
10 }
11
12 // New helper methods

> Details & Provenance (Confidence: Verifying)

Edit instructions for regeneration

Workflow failed after 3 attempts. See the logs below for details.

?

?

Apply

Regenerate

Cancel

Updated

AntiCopyPaster

Refactoring Suggestion

[AntiCopyPaster] Refactoring Suggestion

AntiCopyPaster found a Type-1 clone in this file.

The pasted code in `DuplicateTest#computeStats` starts on line 3 and matches existing code in `DuplicateTest#duplicateMethod` starting on line 20.

Suggested refactoring:

Extract the shared calculation logic into a new helper method named `calculateArrayStats`.

This would replace both duplicate code blocks with calls to the new method, reducing repeated code while keeping the same behavior.

Verification status:

AntiCopyPaster is checking usefulness, isolated compilation, and tests. The Apply button will stay disabled until verification passes.

Diff: Extract Method `calculateArrayStats`

Side-by-side viewer Do not ignore Highlight words 2 differences

Before: Original Code

```
// Target clone methods
// DuplicateTest#computeStats
public static double[] computeStats(int[] data) {
    double sum = 0.0;
    double prod = 1.0;
    int count = data.length;
    for (int i = 0; i < count; i++) {
        sum = sum + data[i];
        prod = prod * data[i];
    }
    double mean = sum / count;
    double variance = 0.0;
```

After: Proposed Refactoring

```
1 // Target clone methods
2 // DuplicateTest#computeStats
3 public static double[] computeStats(int[] data) {
4     return calculateArrayStats(data);
5 }
6
7 // DuplicateTest#duplicateMethod
8 public static double[] duplicateMethod(int[] data) {
9     return calculateArrayStats(data);
10 }
11
12 // New helper method
```

> Details & Provenance (Confidence: Verifying)

Edit instructions for regeneration

Workflow failed after 3 attempts. See the logs below for details.

?

Apply

Regenerate

Cancel

Diff Preview

! UserValidator.java : 42-48

×

```
42 if (u.getUsername() == null || u.getUsername().trim().is
43     throw new IllegalArgumentException("Username is requi
44 }
```

✓ validateInputs(...)

×

```
12 private void validateInputs(User u) {
13     if (u.getUsername() == null || u.getUsername().trim().
14         throw new IllegalArgumentException("Username is requ
15 }
```

AI Trust / Explainability: Learn More View

AI Trust / Explainability — Learn More View

This detailed panel opens AFTER the user clicks "Learn more" from the compact AI Trust section.

1. DETAILED "LEARN MORE" PANEL — STATES

A SUCCESS STATE (All Validations Passed)

AI Trust / Explainability

High Trust

Overall AI Trust Score

95%

High confidence. Safe to apply.

Clone Detection Confidence

95%

Refactoring Agent Confidence

90%

Refactoring Usefulness

90%

Compilation Result

100%

Test / Behavior Preservation

100%

Formula: 20% Clone Detection Confidence + 20% Refactoring Agent Confidence + 20% Refactoring Usefulness + 20% Compilation Result + 20% Test / Behavior Preservation

Rationale

All validations passed with strong confidence. The refactored code compiles successfully and preserves original behavior in tests.

What this means

High trust in this change. Safe to apply after a quick review of the diff and explanation.

B WARNING / PARTIAL STATE (Some Validations Have Issues)

AI Trust / Explainability

Medium Trust

Overall AI Trust Score

65%

Some checks failed. Review before applying.

Clone Detection Confidence

95%

Refactoring Agent Confidence

80%

Refactoring Usefulness

60%

Compilation Result

100%

Test / Behavior Preservation

20%

Formula: 20% Clone Detection Confidence + 20% Refactoring Agent Confidence + 20% Refactoring Usefulness + 20% Compilation Result + 20% Test / Behavior Preservation

Rationale

Some validations scored below the threshold. Specifically, usefulness is moderate and test preservation is low.

What this means

Review the diff and explanation carefully. Consider manual verification before applying this change.

C FAILURE / UNAVAILABLE STATE (Score Cannot Be Calculated)

AI Trust / Explainability

Unavailable

Overall AI Trust Score

Unavailable

Score cannot be calculated.

Clone Detection Confidence

N/A

Refactoring Agent Confidence

N/A

Refactoring Usefulness

N/A

Compilation Result

Failed 0%

Test / Behavior Preservation

Failed 0%

Formula: 20% Clone Detection Confidence + 20% Refactoring Agent Confidence + 20% Refactoring Usefulness + 20% Compilation Result + 20% Test / Behavior Preservation

Reason

Compilation failed due to syntax errors in the refactored code. The score cannot be calculated until this step is resolved.

What to do next

Fix the compilation issues and re-run validation to see the updated trust score and breakdown.

2. IN-CONTEXT USAGE EXAMPLE

[AntiCopyPaster] Refactoring Suggestion

1 Explanation

AntiCopyPaster detected that the code you pasted is a Type-2 clone of the existing method in the same class.

- Existing clone location: UserValidator.java:42-48
- Pasted clone location: UserValidator.java:101-107
- Suggested method name: validateInput(...)

The assistant suggests extracting the common validation logic into a new method and replacing the duplicate with a method call.

2 Diff Preview (Preview)

Original (Duplicate) Code

```
if (u.getUserName() == null ||
    u.getExternalID().isEmpty()) {
    throw new IllegalArgumentException(
        "Username is required");
}
if (u.getEmail() == null ||
    EMAIL_PATTERN.matcher(u.getEmail())
    .matches() { ... }
```

Extracted (Replacement)

```
+ validateInput(u, EMAIL_PATTERN);
+ ...
```

3 AI Trust / Validation

Learn more →

95%

AI Trust Score

95%

90%

90%

100%

100%

Formula: 20% each across five workflow stages.

4 Actions

Apply

Reject

Preview Diff

...

Clicking "Learn more" opens the detailed AI Trust / Explainability view (this panel).

Buttons

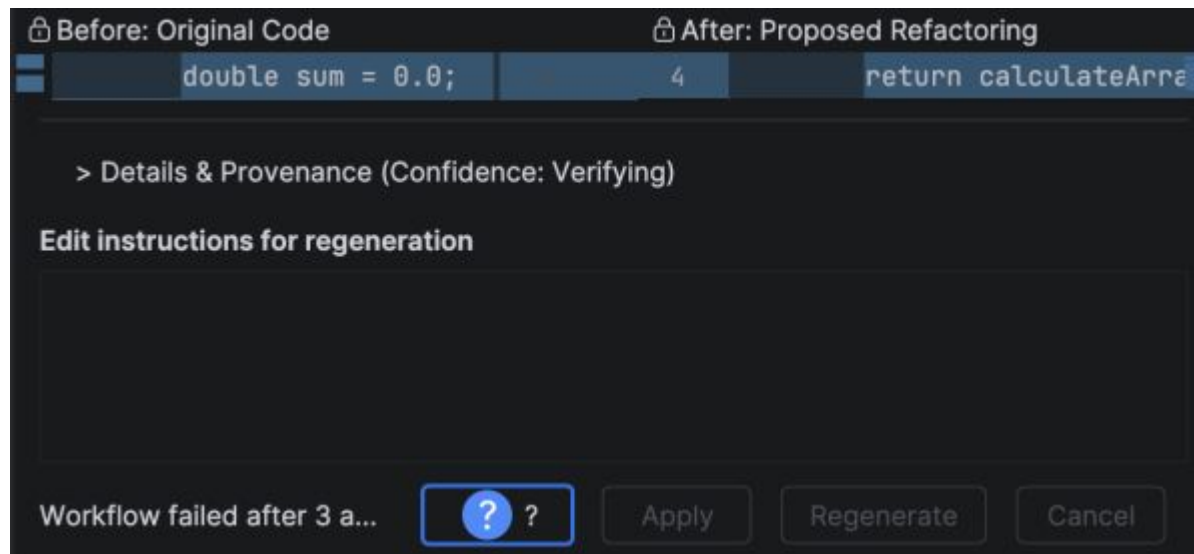
Individual Acceptance

The user should have the opportunity to apply, reject, or view details for each clone instance

Disabled Acceptance

The apply button should be disabled in the case that validation fails, as there will be nothing to apply.

Original



v Details & Provenance (Confidence: Verifying)

Clone Type: Type-1 (same code except for whitespace or comments)
Source Location: [DuplicateTest.java:20 \(DuplicateTest#duplicateMethod\)](#)
Pasted Location: [DuplicateTest.java:3 \(DuplicateTest#computeStats\)](#)
Clone ID: c1_selected
Target Methods: DuplicateTest#computeStats <-> DuplicateTest#duplicateMethod
Suggested Refactor: extracted_method
Selected Ranges: 3-18, 20-35
Detection Reason: The code block inside computeStats is identical to the code block inside duplicateMethod, performing the same statistical calculations on the input array.
Verification: Usefulness check, isolated compilation, and tests are running.

Edit instructions for regeneration

Workflow successful.

Apply

Reject

Details

Editor View

v Details & Provenance (Confidence: Verifying)

Clone Type: Type-1 (same code except for whitespace or comments)
Source Location: [DuplicateTest.java:20 \(DuplicateTest#duplicateMethod\)](#)
Pasted Location: [DuplicateTest.java:3 \(DuplicateTest#computeStats\)](#)
Clone ID: c1_selected
Target Methods: DuplicateTest#computeStats <-> DuplicateTest#duplicateMethod
Suggested Refactor: extracted_method
Selected Ranges: 3-18, 20-35
Detection Reason: The code block inside computeStats is identical to the code block inside duplicateMethod, performing the same statistical calculations on the input array.
Verification: Usefulness check, isolated compilation, and tests are running.

Edit instructions for regeneration

Workflow unsuccessful.

Apply

Reject

Details

Editor View

Next Steps

Pull Requests

When finished, push your branch and open a pull request into the ui_panel branch.

Team Review

Review changes, fix conflicts, then merge accepted pull requests into ui_panel.

Keep updating

Keep the rest of the team updated sending screenshots of your work throughout the week.
Update the website content.